

GUITAR HERO EMBEDDED PROJECT

C. Pulido | Clarke University Computer Science Dept.

Guitar Hero Embedded Project Overview

- This is the documentation for my Guitar Hero Embedded Project (GH1) that I designed and built to fulfill my capstone project requirement.
- GH1 is a portable game system; it is a Guitar Hero style arcade rhythm game running on an STM32 Micro Controller (GH1-a) and displayed on an ILI9341 Display. GH1 plays audio by streaming a .wav file from an onboard microSD card. The microSD card is inserted into GH1-a's MicroSD Slot. Five columns feed colored notes down the screen in sync with a song; the player has to hit the corresponding note with precise timing to achieve the maximum score. The five colored arcade LED buttons have colors that correspond to the color of notes on the game.
- GH1 is built from the STM32F407ZGT6 M4 Development board from AliExpress. There is no documentation for this board, but it follows many conventions from other STM32F4 boards. I've labeled this board as 'GH1-a' for my own purposes. GH1a is a microcontroller development board: a ready-to-use circuit board built around a small, low-power computer chip. It includes all the essential support components required to power, program, and interact with the chip directly, allowing you to prototype hardware projects without building a circuit board from scratch.
- GH1-a connects to a custom 2-layer PCB for routing button inputs (GH1-b). I designed GH1-b with kiCAD, kiCAD is a free, open-source software suite for Electronic Design Automation (EDA). It is used by engineers, hobbyists, and professionals to design electrical schematics and create layout designs for printed circuit boards (PCBs) and had it printed through PCBway, PCBway is a China-based manufacturer specializing in PCB (Printed Circuit Board) fabrication.
- GH1's housing is comprised of 5 3D-printed PLA pieces. I designed all these pieces in FreeCAD: an open-source parametric 3D modeler used to design real-life objects. The 3D printer I used at my college to print all the parts was the Bambu LAB A1.
- Firmware was written in STM32CubeIDE and flashed to GH1-a using a USB STLINKV2 device. The STLINKV2 I used was likely a Chinese clone and not OEM from STMicroelectronics.

Contents

Guitar Hero Embedded Project Overview	1
Contents.....	2
Peripherals	4
GPIO	4
DAC / Audio Output	4
SPI / Display	4
SDIO / MicroSD Song Storage	5
Components.....	5
1. GH1-a.....	5
2. GH1-b.....	5
3. DC 12V/24V to 5v10A Buck Converter.....	6
4. 14.8v LiPo Battery 1550mAh 100C OVONIC High Discharge FPV 451p.....	6
5. 5x JZT-08 Arcade Style 30mm Buttons.....	6
6. XT60 Connector with 3" Wire Leads	7
7. Hexagonal Plastic Nut from the JZT-08 buttons	7
8. XT60 Power Switch.....	7
9. 2x 3W 8Ω ALMOCN Full-Range Speakers	8
10. PAM8403 Amplifier Board	8
11. Hiletgo ILI9341 2.8" SPI TFT LCD Display	9
12. PLA 3D Printed Components	9
CAD Drawings	13
Guitar Body Drawing	13
Guitar Body Faceplate Drawing.....	15
Neck Body Drawing	17
Neck Body Faceplate Drawing	18
Neck Body Union Piece Drawing	19
GH1 Assembly Drawing.....	20
GH1 Schematic.....	23
.....	23
April 2026	2

GH1-b.....	24
GH1-b Schematic.....	26
Flashing	27
Pinout and Configuration	28
System Configuration	31
Clock Configuration.....	33
Physical Internal Layout.....	34
.....	35
Song and MicroSD Card	37
Software Architecture	38

Peripherals

GH1-a Peripherals

1. Input: GPIO → GH1-b → 5 JZT-08 30mm Arcade Style Buttons
2. Audio: DAC --> PAM8403 Amplifier Board --> 2x 3W 4 Ω ALMOCN Full-Range Speakers,
3. Display: SPI --> ILI9341
4. Song Storage: SDIO- -> MicroSD Card

GPIO

The five arcade buttons slide into holes on the neck faceplate of the guitar. Inside the necks housing, XH2.5mm male to male wires run from the bottom XH2.5 header on each button to their corresponding socket on my GH1-b custom PCB. From GH1-b, jumper wires run from one of the five pins in a female socket to the corresponding GPIO pin on GH1-a. The GPIO peripheral polls for when one of the five pins are pulled high and registers that input in the game logic.

DAC / Audio Output

The audio peripheral is complex. It encompasses a DMA channel that streams bytes from SDIO to the DAC and then the DAC outputs the analog waveform on pin PA4 (Labeled DAC_OUT1 in CubeIDE) to the PAM83403D amplifier board. The PAM83403D amplifier board is a Chinese audio amplifier board from amazon, It has the PAM8403D IC on it.

PCM audio is read from the microSD card via SDIO DMA into a double buffer and streamed to the DAC for continuous playback. I don't understand it well, but double buffering is standard practice for many embedded DMA applications.

The PAM8403D Amplifier Board is wired to two 3W 4 Ω ALMOCN speakers. Each speaker has a + and - wire that run to a + and – terminal on the board. The amplifier board gets power from the +5v terminal on the main 5v buck converter. The boards' GND pin is connected to one of GH1-a's GND pins.

SPI / Display

The display peripheral is an SPI output to an ILI9341 TFT LCD module where GH1-a acts as the master, and the LCD acts as the slave. In my firmware, I'm using an ILI9341 display driver with some basic drawing functionality to communicate with the LCD. The driver abstracts a lot of low-level functionality and lets me easily draw rectangles, text, pixels, clear the screen, etc.

There are five dedicated SPI connections that run from GH1-a to the display module: a clock line (SPI1_SCK), a Data/Command line (SPI_DC) that designates whether data or a command is being sent to the slave device, a reset line (SPI_RESET) which is very important because the ILI9341 frequently has to be reset for proper functioning, a chip-select line

(SPI_CS) that is pulled low by the driver to designate which slave is currently selected, and a master-out-slave-in (SPI1_MOSI) that transmits the serial RGB pixel data to the ILI9341.

For powering the display: there is an LED backlight pin on the display that is tied to a +3v3 output on GH1-a, a VCC pin for the display that is tied to another +3v3 output on GH1-a, and a GND pin that is tied to a GND pin on GH1-a.

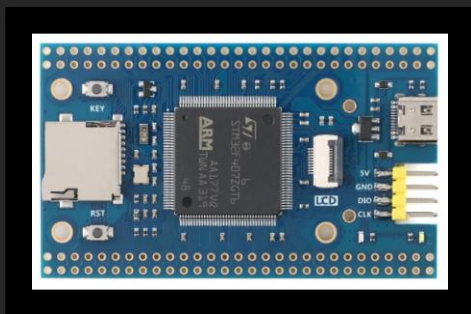
SDIO / MicroSD Song Storage

The SDIO peripheral interfaces with a MicroSD card used to store the song file. During gameplay, the microcontroller reads song information from the card and loads it into memory as needed using double buffered DMA. This allows a large song library to be stored without consuming the GH1-a's internal flash memory.

Components

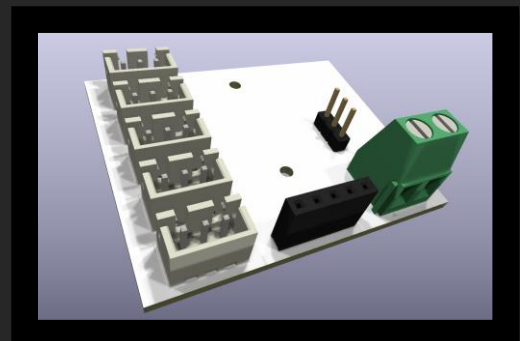
GH1 Components

1. **GH1-a: STM32F407ZGT6 M4**
Development board (AliExpress)
The microcontroller. GH1-a is located in the center of the guitar body. It's glued on top of a hexagonal plastic nut (Component 7). For GH1, the male header rows are soldered onto the top of GH1-a and facing up so jumper cables can be easily attached and swapped around.



2. **GH1-b: Button Routing Auxiliary**
PCB (Custom)

GH1-b is a custom PCB for organizing/routing wire connections from the MCU to the buttons. Without the GH1-b, the connections would be messy and un-organized. It sits on top of 4 standoffs in the Guitar body just under where the neck of the guitar attaches. It's located there as a midpoint between GH1-a and the arcade buttons that line the guitar neck.



3. DC 12V/24V to 5V10A Buck Converter

This is a 5V10A buck converter I got off amazon to regulate the large unstable voltage from the LiPo battery. The buck converter is super glued to the floor of the guitar body at the opposite corner of the guitar body from the battery. There are two big Phillip's screw terminals for input and another two terminals for output. The wire leads (Component 6) are electrically connected by being clamped into the input

terminals. Many of the GNDs of other components run back to the negative terminal, and the +5V terminal is where GH1-a, GH1-b, and the Audio Amplifier board get power from.



4. 14.8v LiPo Battery 1550mAh 100C OVONIC High Discharge FPV 451p

I chose to use a LiPo battery as opposed to alkaline batteries for the high potential current draw of the device. Using alkaline batteries: AAA, AA, or 9V would severely limit the number of Amps I could pull for all the components I wanted. These LiPo type drone batteries allow me to have an overhead with how much components I wanted to add.



5. 5x JZT-08 Arcade Style 30mm Buttons (White, Yellow, Red, Green, and Blue)

The arcade buttons came in a kit off amazon that included two of each color and an arcade joystick that I did not use. The name of the listing on amazon was "EG STARTS Arcade Buttons 1 Player DIY Kit Joystick 5V LED Arcade Button for Arcade Stick PC Games Mame Raspberry pi". The buttons use a microswitch that is a standard SPST-NO momentary microswitch (Single Pole, Single Throw, Normally Open). When a button is pressed, the switch contacts close, causing the corresponding GPIO input to change state and allowing the microcontroller to detect the button press. The buttons also contain integrated LEDs that illuminate when the buttons are pressed.



6. XT60 Connector with 3" Wire Leads 10ga insulated wire

These two leads are run from a female XT60 connector to the + and GND input screw terminal on the 5v buck-converter. The main power switch is connected to these leads via the XT60 connector.



7. Hexagonal Plastic Nut from the JZT-08 buttons

Used as a standoff for GH1-a. Ten of these came with the kit that included the JZT-08 arcade buttons (Component 5). The arcade buttons have threads at the bottom of them. The threads themselves aren't used because the buttons are held into the Neck Body Faceplate' holes with friction and superglue. I used one of these hexagonal nuts as a standoff for GH1-a inside the Guitar Body (12-a). The nut was placed in the center of the Guitar Body and glued

to the bottom. The bottom of GH1-a is glued to the nut. The superglue I used was non-conductive, so it worked out fine. I would've reprinted the Guitar Body piece and properly lined up the already existing standoffs for GH1-a. That was my original intention. Bad tolerances that required reprints and time limits caused me to make the decision to use this component instead.



8. XT60 Power Switch

The power switch enables convenient toggling between on and off states. The power switch is a Single Pole Single Throw switch between two XT60 Female connectors and is located on the faceplate. One female XT60 connector from the switch connects to the battery (Component 4) and the other female XT60 connector from the switch connects to the male XT60 connector on the XT60 Wire

leads (Component 6).



9. 2x 3W 8Ω ALMOCN Full-Range Speakers

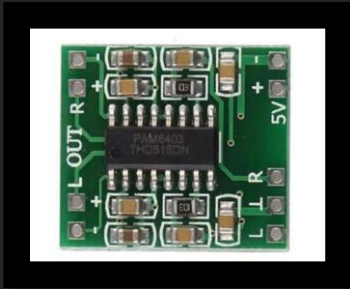
These speakers have two protruding triangles from a square body and fit into recesses on Guitar Body Faceplate. They are Chinese and I couldn't find their proper name or documentation so here are some specifications from the Amazon listing:

3W max output per channel, 4Ω/8Ω impedance, 500-20kHz frequency response, 84dB sensitivity

The connectors on the ending of the wire leads need to be snipped off and the endings need to be stripped so they can be soldered to the amplifier board. I believe it's 20ga wire. There is a cutout in the faceplate where these specific speakers fit into and perfectly mesh with the surface of the faceplate (Component 12-b). They are adhered to the faceplate with 3D printing superglue.

10. PAM8403 Amplifier Board

The PAM8403 Amplifier board is a Chinese board with a PAM8403 IC on it. The board features Dual-channel stereo, 5V power supply can output 3W +3 W power, it can directly drive 4Ω, 8Ω small speakers. In GH1 a single PAM8403 Amplifier board is connected to pin PA6 (DAC_OUT1) and is fed the analog audio signal from the DAC. The analog signal from the DAC is routed to both the left and right audio input channels of the amplifier. The amplification power is fed to the Amplifier board by connecting the +5v and GND terminals on the amplifier board to the output terminals +5v and GND on the 5v10a buck-converter (Component 3). The signal ground terminal on the board runs back to one of GH1-a's GND pins. See GH1 Schematic for additional wiring details



11. Hiletgo ILI9341 2.8" SPI TFT LCD Display Touch Panel 240X320 with PCB 5V/3.3v

You can find these anywhere. The one I used in GH1 was manufactured by HiLetgo. They are abundant on amazon and AliExpress. The faceplate (Component 12-b) has a slot that the display module fits into, the pins on the LCD are exposed to the interior of the guitar body so they can connect to GH1-a via female-to-female jumper wires.



12. PLA 3D Printed Components

GH1 Housing is split into five 3D printed PLA components; the most important is the body because it's where the bulk of other components are placed. The Guitar Body has six 10mm $\varnothing$ rods that fit into protruding rings on the underside of the Guitar Body Faceplate. After adjusting tolerances multiple times, I found that the best

friction fit was making the protruding rings 0.5mm bigger than the diameter of the rods. The Guitar Body also has standoffs for GH1-a and GHA-b. The standoffs in the center for GH1-a did not end up being used; I ended up using Component 7 in the center of the four central standoffs instead. There are two standoffs near the neck area in the Guitar Body where GH1-b is superglued to.

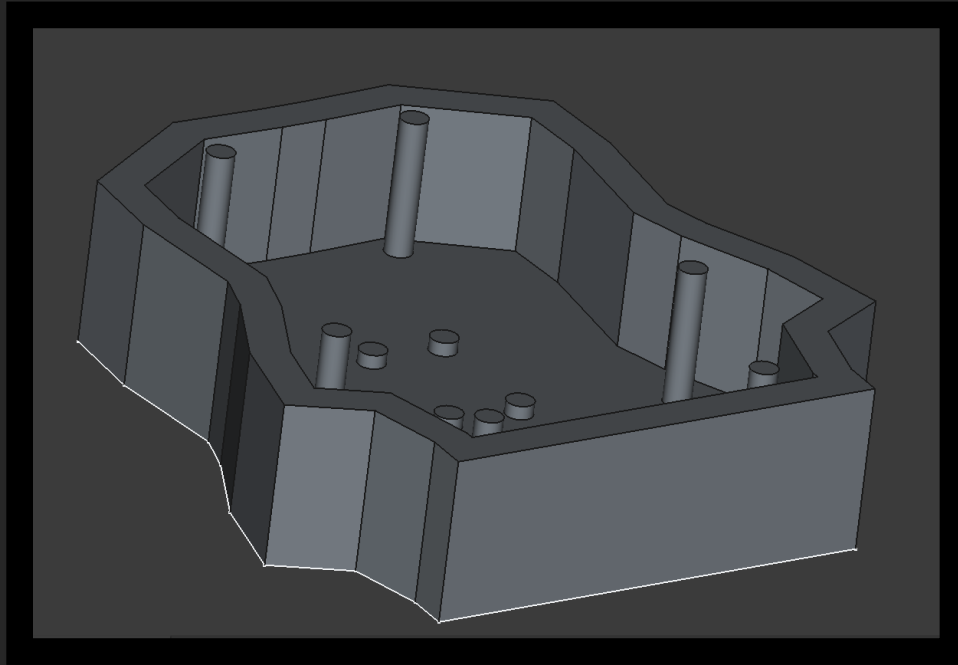
The Neck Body (12-c) and Neck Body Union Piece (12-e) need to be superglued together first with 3D Printing Superglue.

At the top flat surface of the Guitar Body (12-a), four 10.7mm holes need to be drilled so that the Neck Body Union Piece (12-e) can fit in with its four corresponding rods.

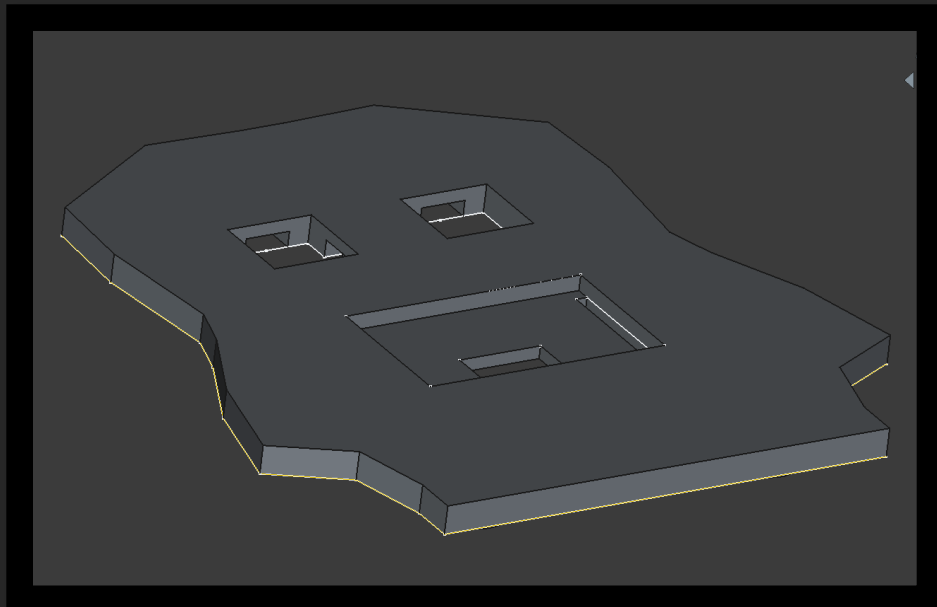
3D printing superglue should be applied generously to all adhering surfaces during the joining of the Guitar Body (12-a) and Neck Body Union Piece (12-e). Before Joining the Guitar Body and Neck Body Union Piece, make sure the Neck Body Union Piece and Neck Body are already be glued together and dried. The top edges of the Neck Body Union and Guitar Body should be flush with each other. After they are connected and the glue is dried, a hole should be drilled or melted with a soldering iron in the center so that the two holes connect and the wires to the arcade buttons can be fed through. The size of the hole in the Guitar Body should roughly match the hole or be slightly smaller than the hole in the Neck Body Union Piece. Do not make it too big

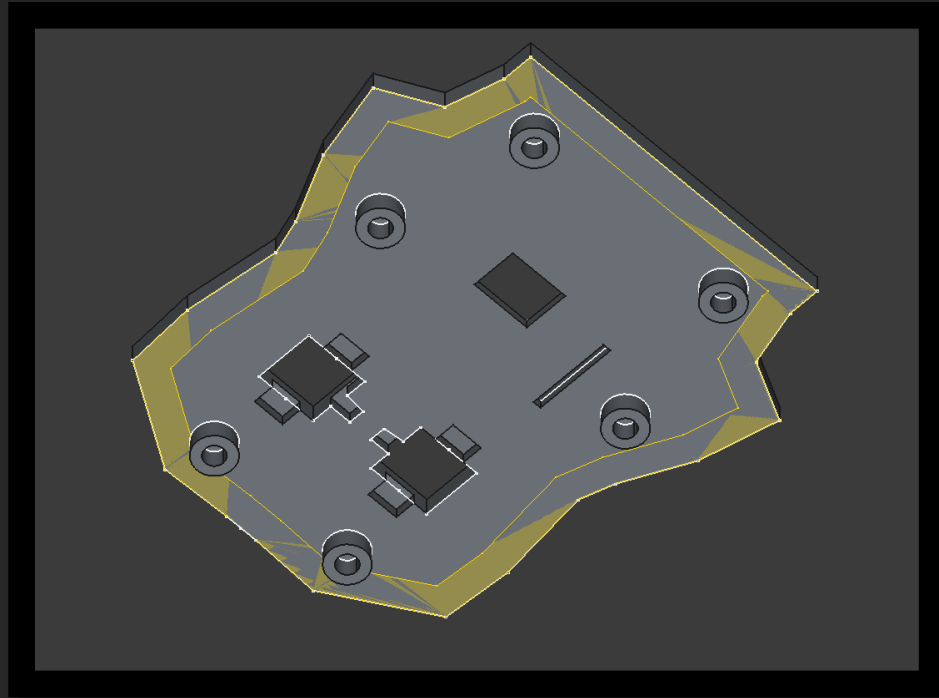
or you will significantly weaken the
Guitars structure

a. *Guitar Body*

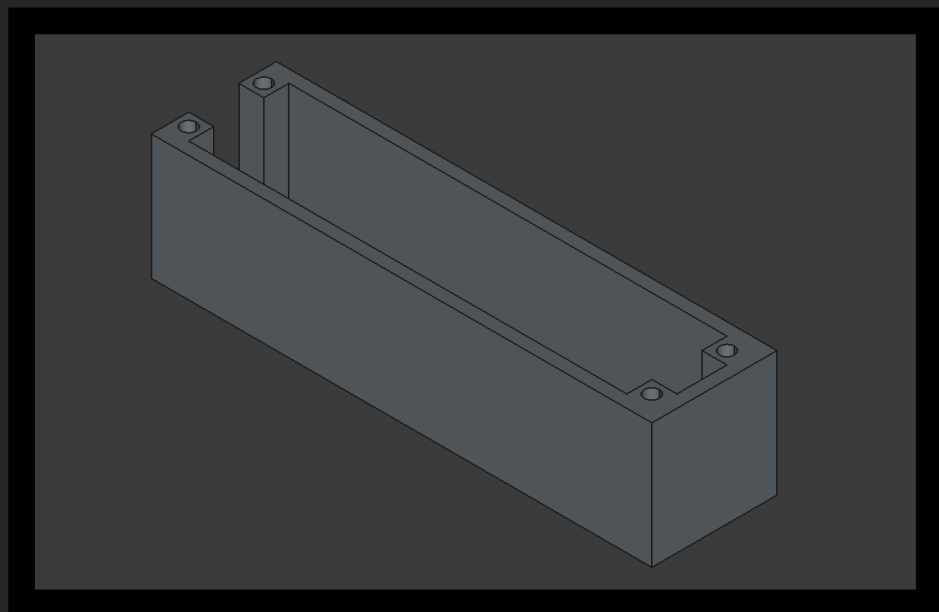


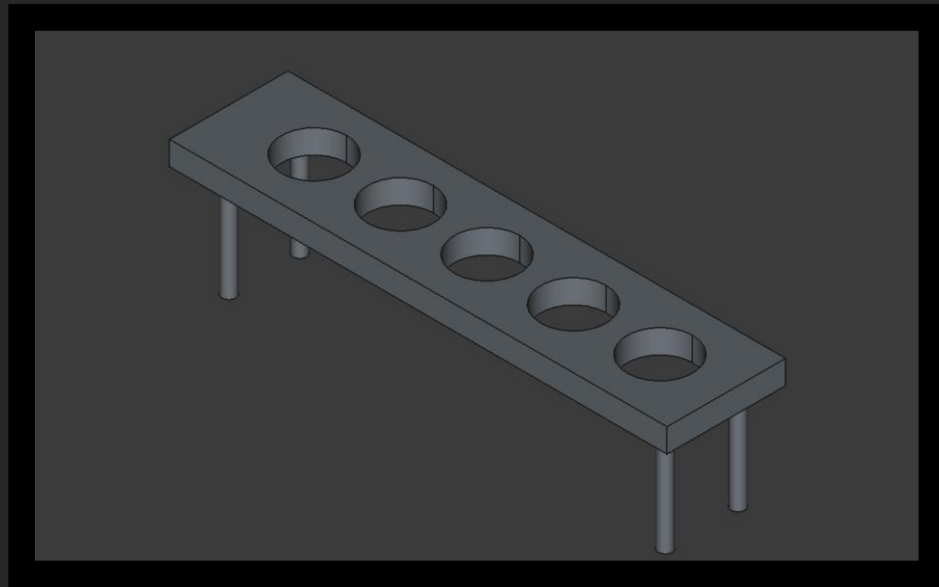
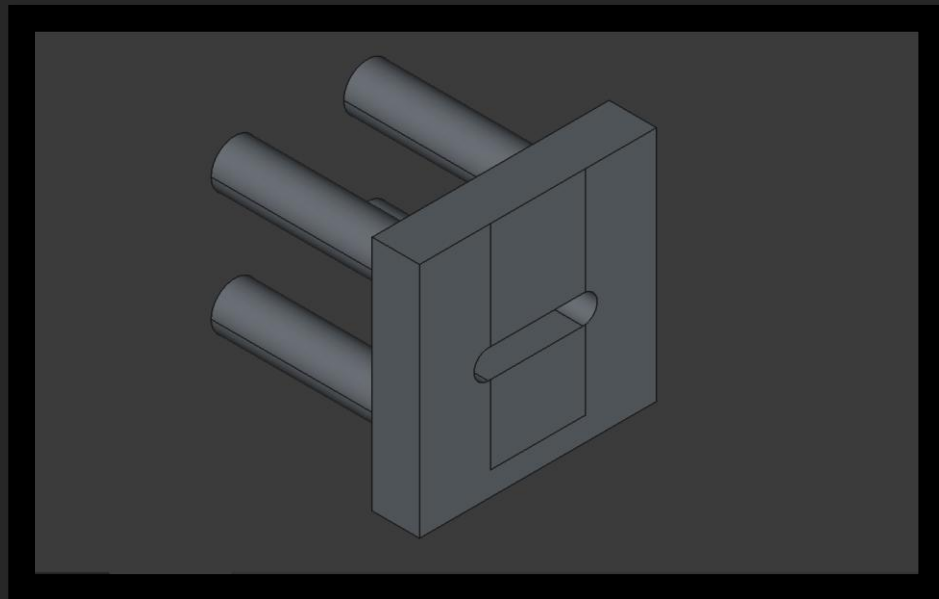
b. *Guitar Body Faceplate*





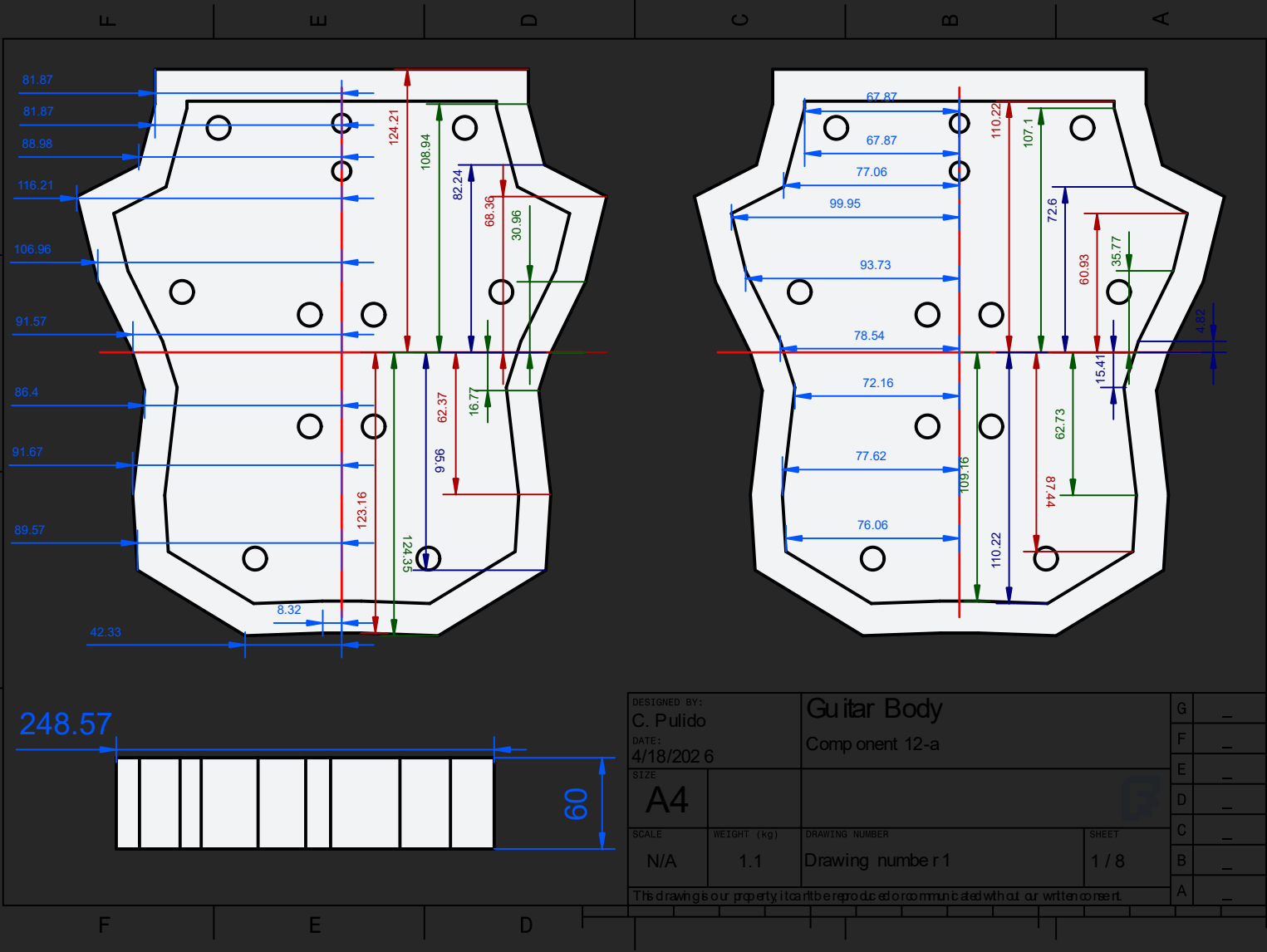
c. *Neck Body*

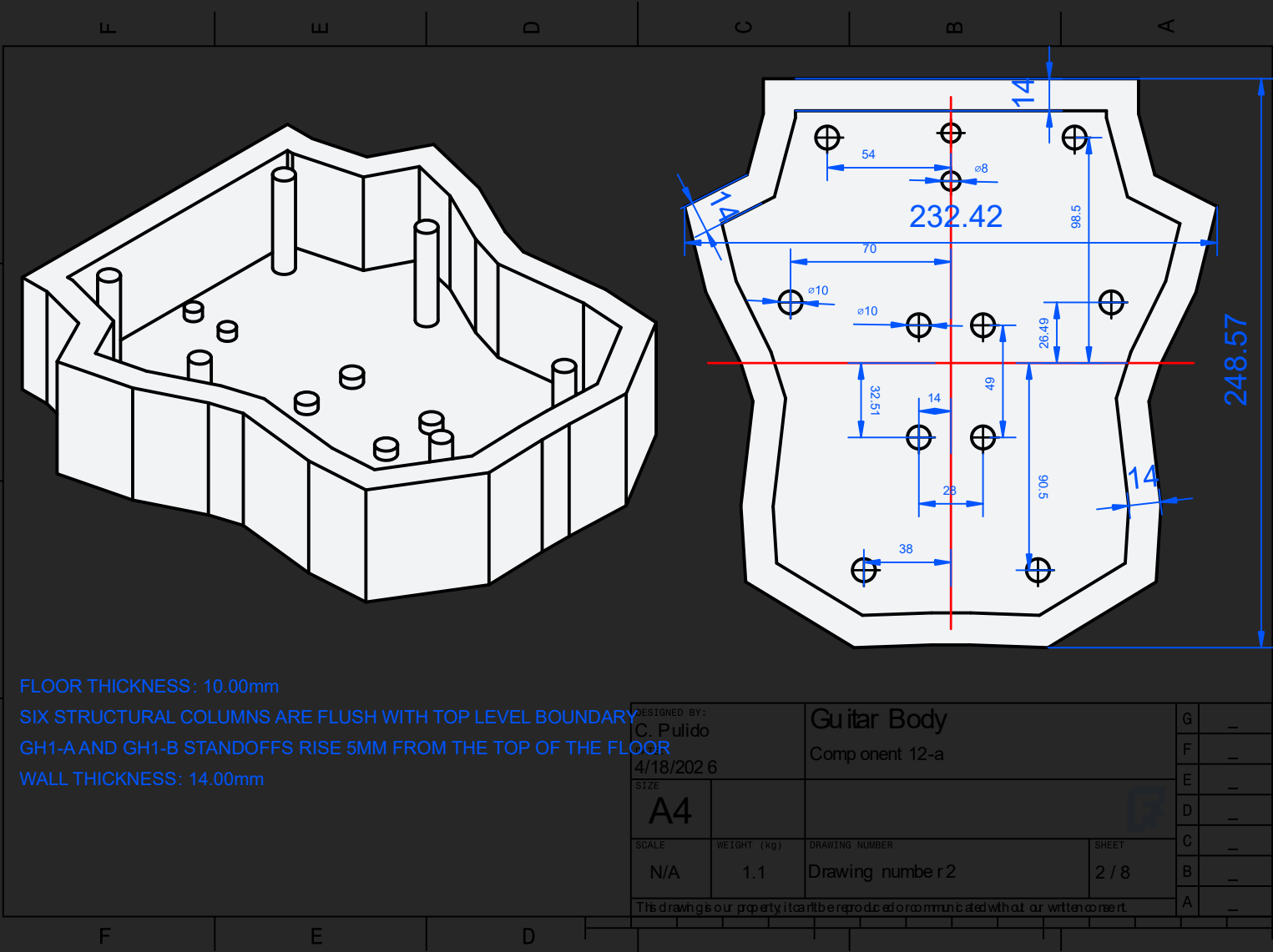


d. *Neck Body Faceplate*e. *Neck Body Union Piece*

CAD Drawings

Guitar Body Drawing





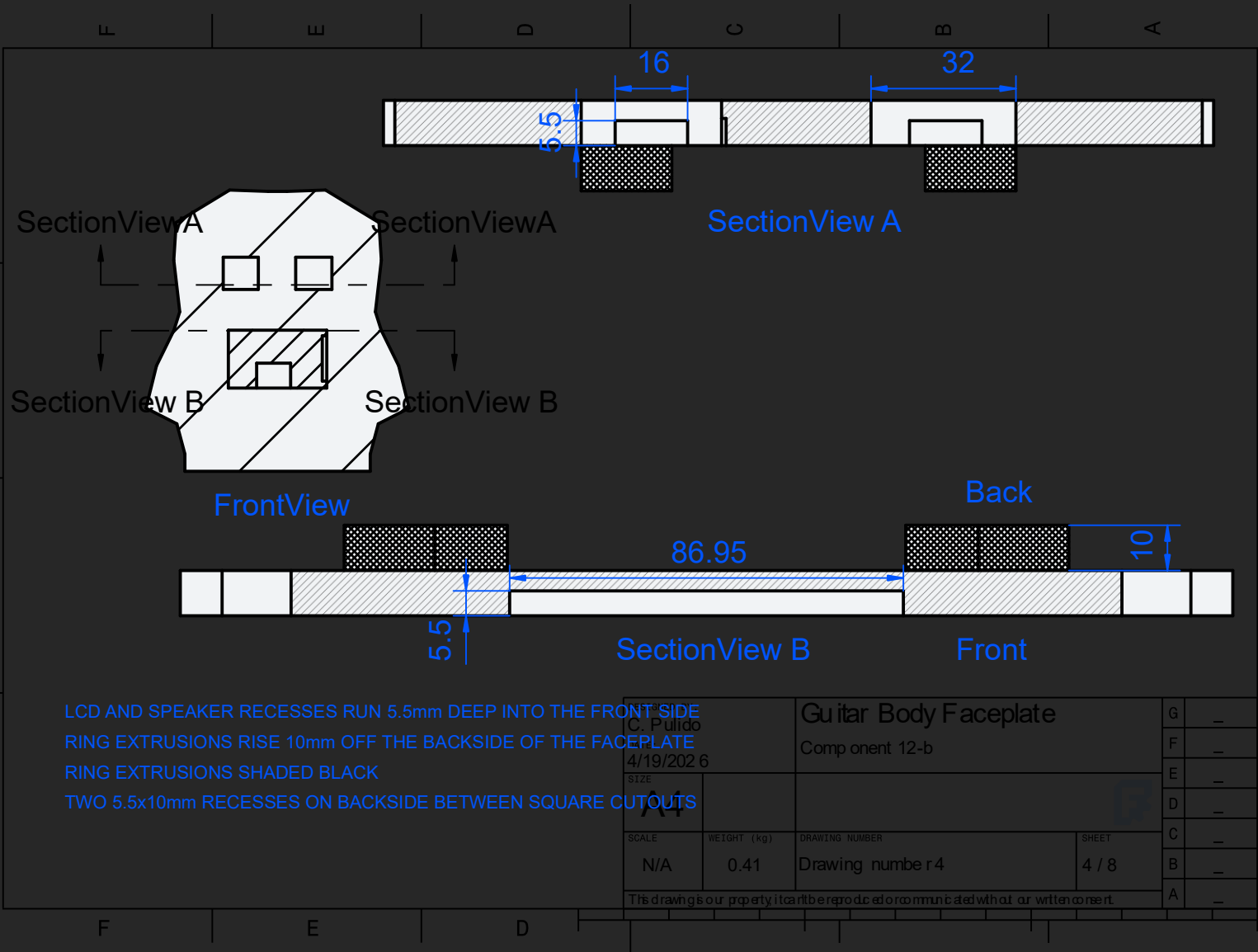
The drawing consists of three views of a guitar body faceplate:

- Top View (Left):** Shows the overall shape with a central rectangular area containing a cross-hatched pattern. Dimensions include 31, 15.99, 86.95, 36.44, 51.15, 28.71, and 30.
- Front View (Right):** Shows the faceplate with various mounting holes and slots. Dimensions include 10, 11, 16, 26.44, 47.44, 4.56, 40, 4, 28.5, 36.44, 47.96, 5, and hole diameters of $\phi 10.5$ and $\phi 20$.
- Side View (Bottom):** Shows the profile of the faceplate. Dimensions include 248.57, 10, 10, and 20.

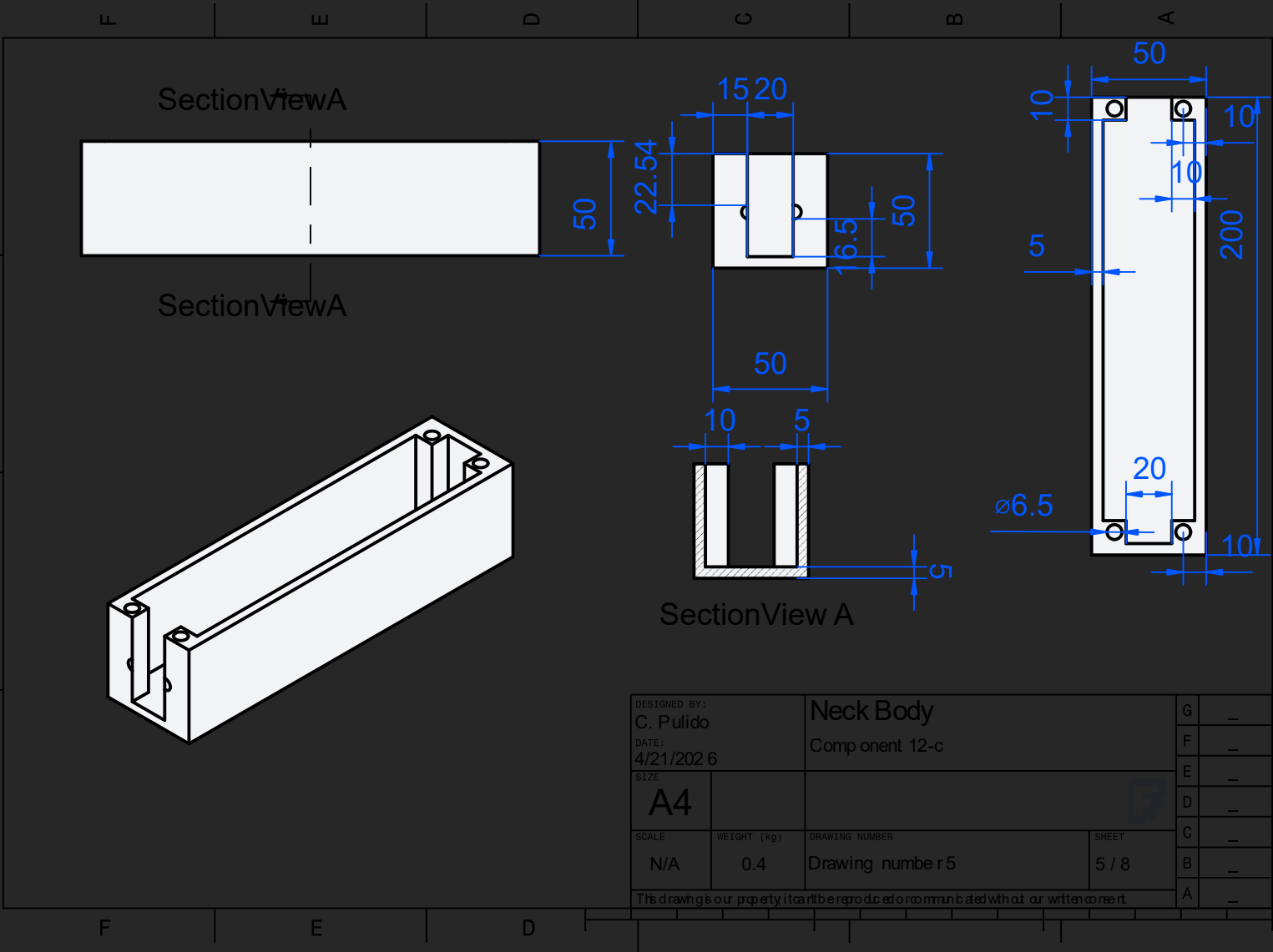
The drawing is titled "Guitar Body Faceplate" and "Component 12-b". It includes a title block with the following information:

DESIGNED BY:	C. Pulido		
DATE:	4/18/2022		
SIZE:	A4		
SCALE:	WEIGHT (kg)	DRAWING NUMBER	SHEET
N/A	0.41	Drawing number 3	3 / 8

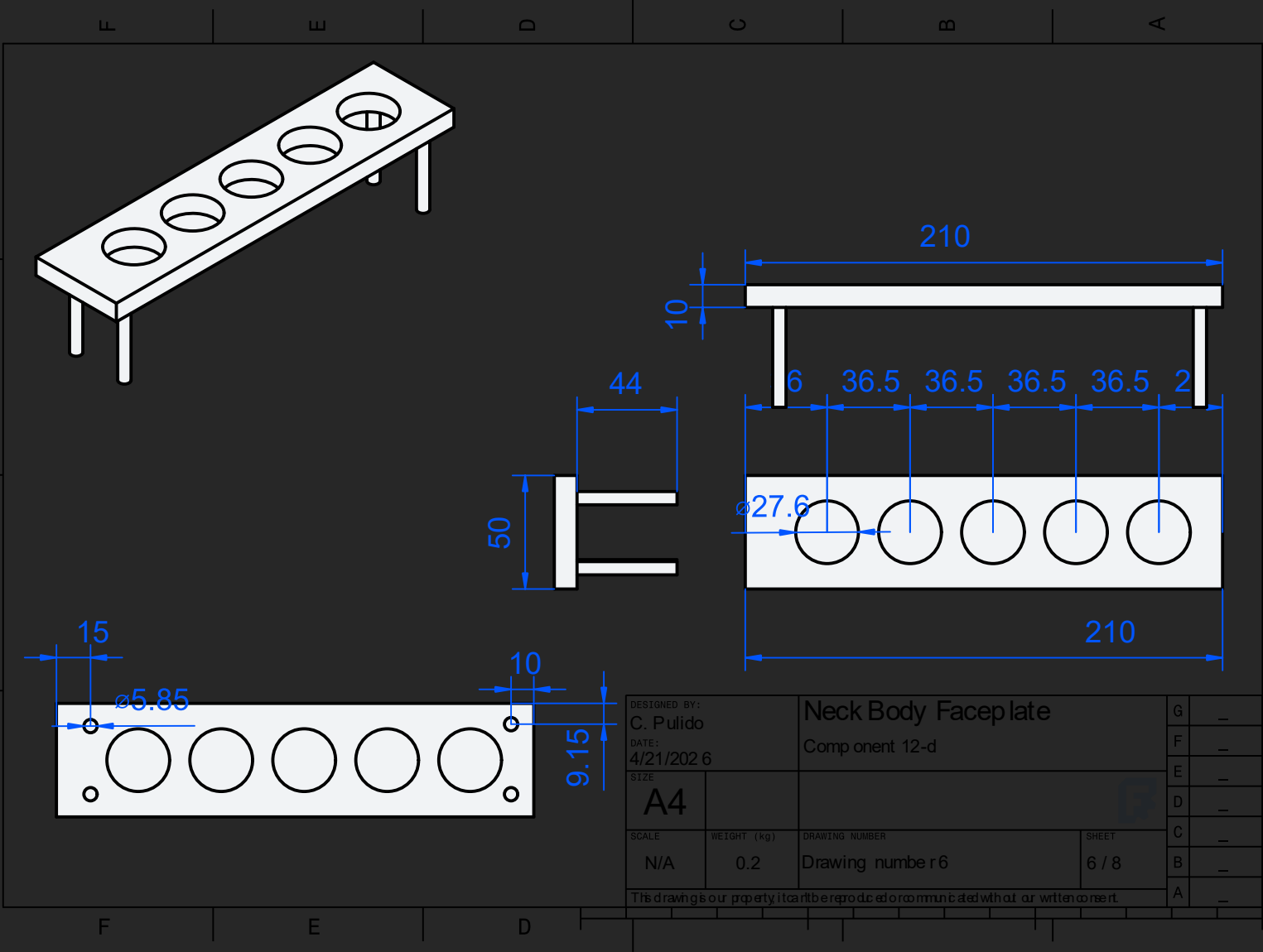
This drawing is our property and it can be reproduced or communicated without our written consent.



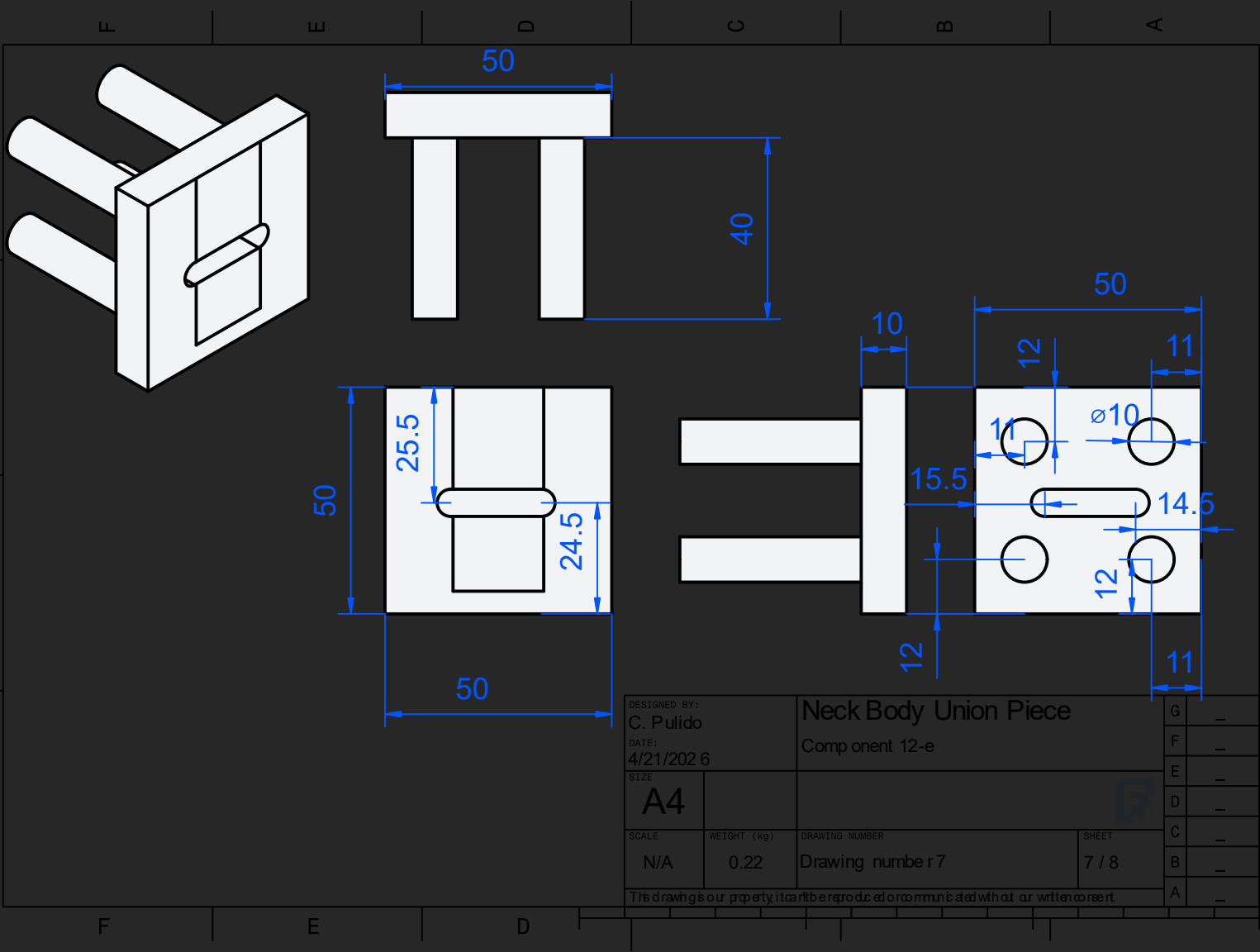
Neck Body Drawing



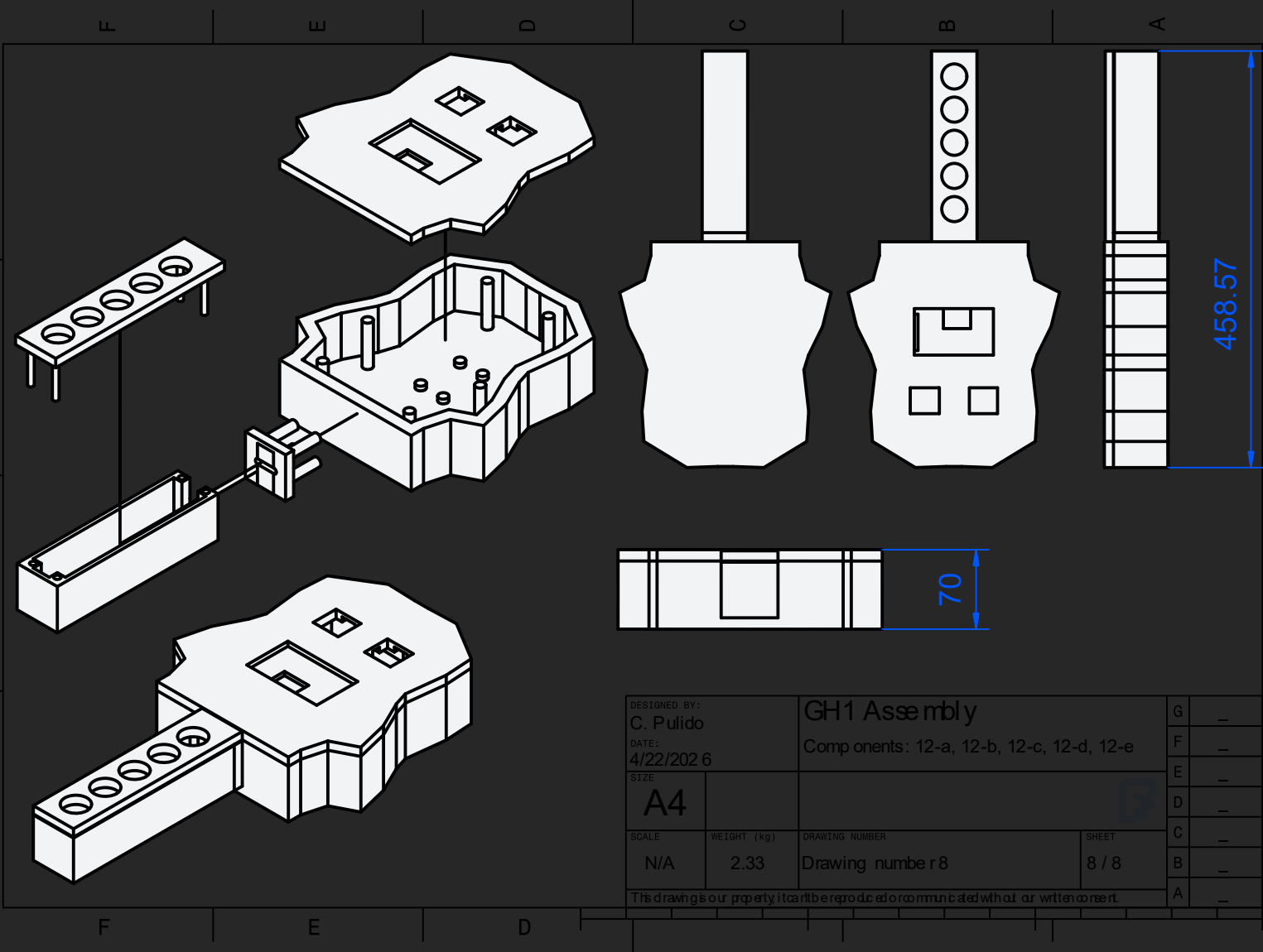
Neck Body Faceplate Drawing



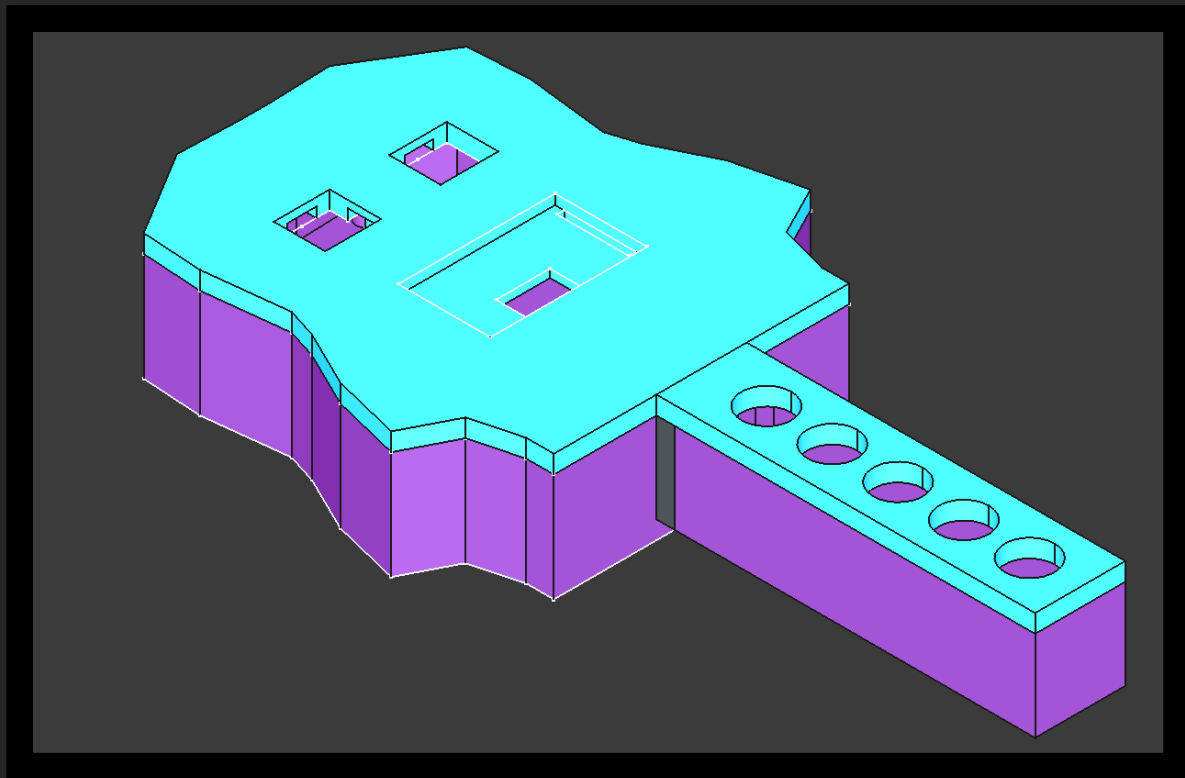
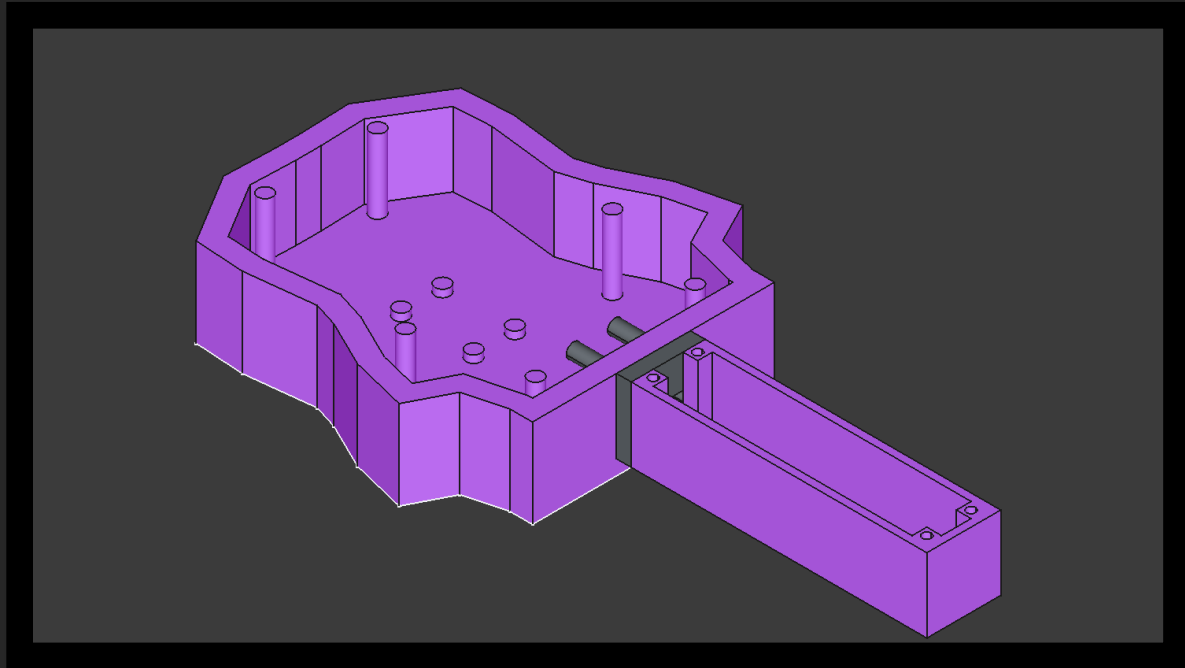
Neck Body Union Piece Drawing

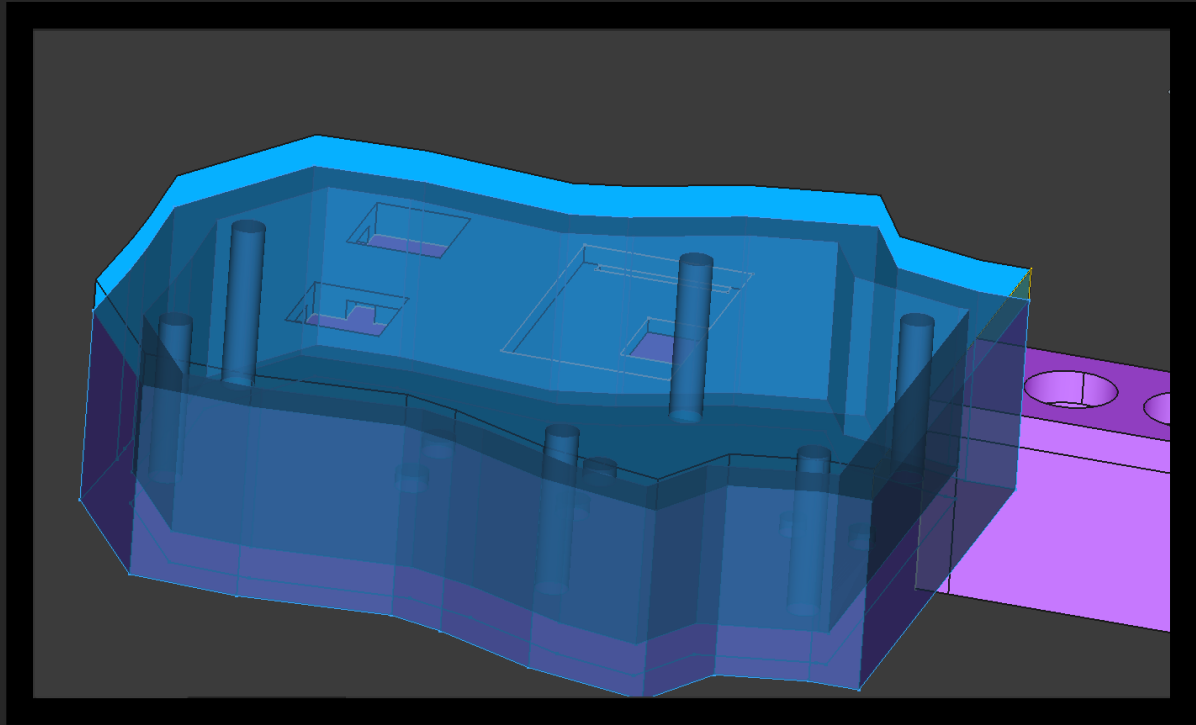


GH1 Assembly Drawing

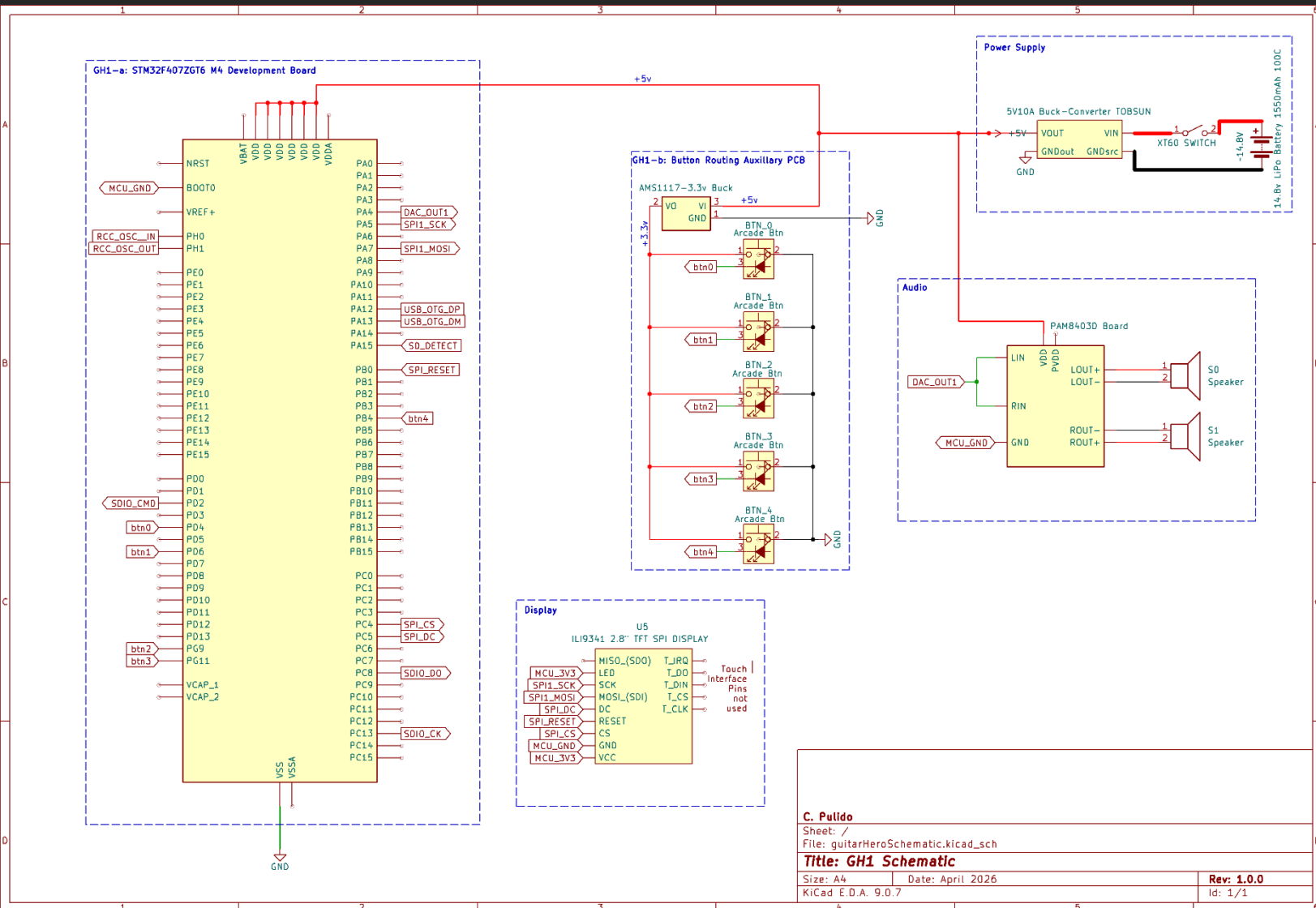


DESIGNED BY: C. Pulido		GH1 Assembly Components: 12-a, 12-b, 12-c, 12-d, 12-e		G	—
DATE: 4/22/2026				F	—
SIZE	A4			E	—
				D	—
SCALE	WEIGHT (kg)	DRAWING NUMBER		C	—
N/A	2.33	Drawing number 8		B	—
		SHEET		A	—
		8 / 8			
This drawing is our property it can't be reproduced or communicated without our written consent.					





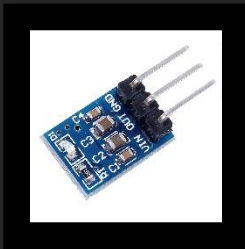
GH1 Schematic



GH1-b

Two Layer PCB routing board uses an onboard buck-converter (HiLetgo-AMS1117-DC-DC-Stepdown-Module-3.3v) to step down +5v to +3v3 as a safer operating voltage for the JST-OH (Component 5) buttons. +5v and GND are connected to + and – on 1715721 Phoenix Contact Screw Terminal. The five-pin header is a S6103-ND (manufacturer part number PPTC051LFBN-RC) is a 5-position, 0.1-inch (2.54mm) pitch male pin header). Five JST-XH 2.54 Male Terminals allow JST-XH ribbon cables to connect to each of the five buttons

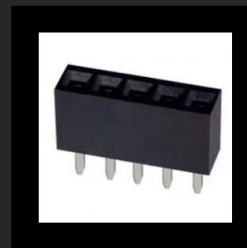
1. HiLetgo AMS1117 DC-DC Step-Down Module (3.3V regulator)
Used to convert +5V input from the power rail into a regulated +3.3V output. This provides a safer operating voltage for the JST-OH button inputs (Component 5).



2. Phoenix Contact 1715721 Screw Terminal Block
Provides the main power input interface to GH1-a. +5V and GND from main buck converter (Component 3) connect to the + and – terminals respectively.



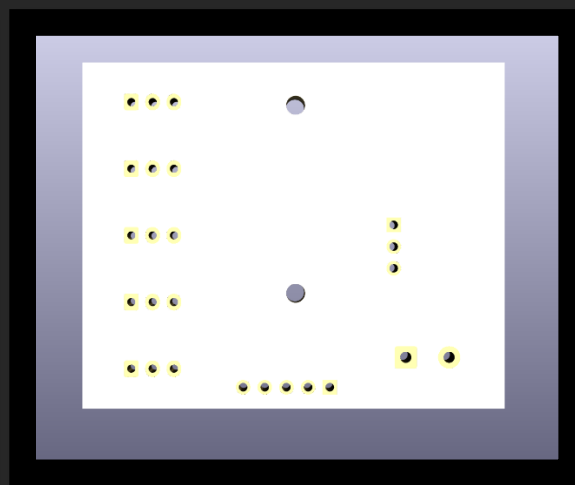
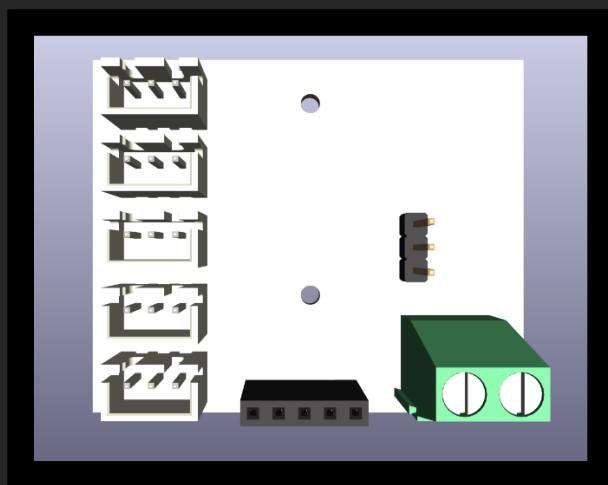
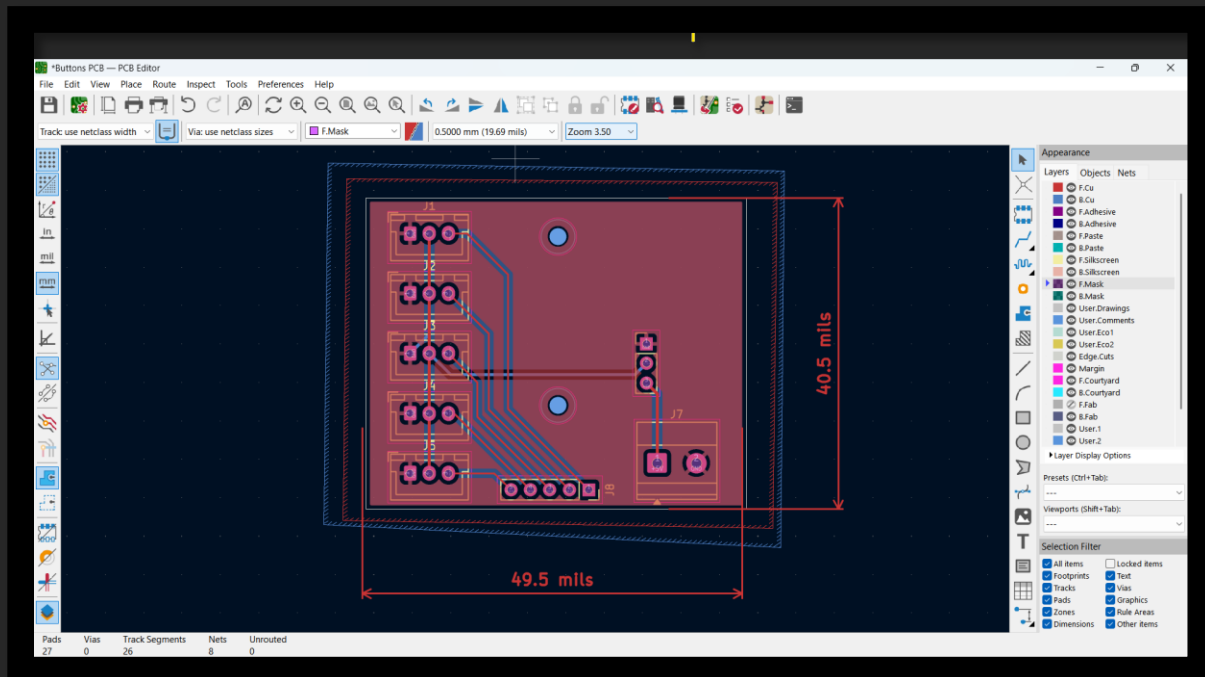
3. S6103-ND (PPTC051LFBN-RC)
5-Position 2.54mm Male Pin Header
Serves as the signal breakout connector for routing the button signals between the GH1-b and GH1-a.



4. 5x JST-XH 2.54mm Male Terminals
Individual connectors used for each of the five arcade button signal lines, allows ribbon cable connections between the buttons and the PCB.



GH1-b Auxiliary Routing Board in KiCad 9.0 Software



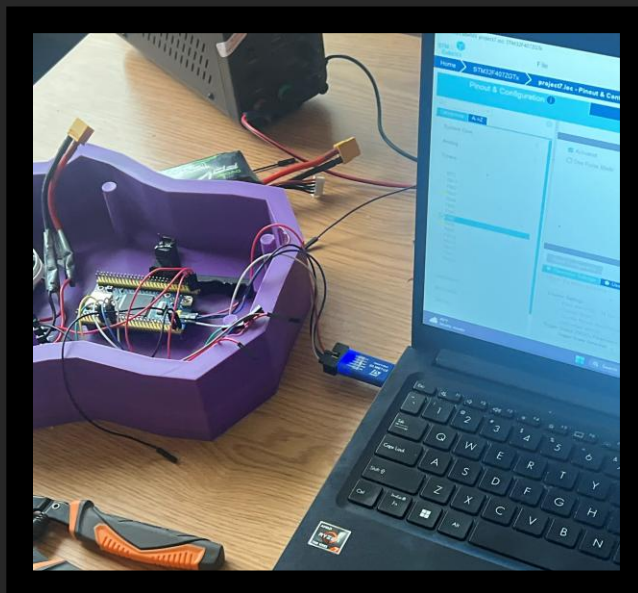
April 2026



Flashing

The project was developed using STM32CubeIDE and STM32CubeMX. To program the board, connect an ST-Link V2 programmer to the SWD pins on the STM32 and connect the programmer to a Windows PC via USB.

1. Download the project file from my git repository 'gh1-project' and unzip
2. Open the project in STM32CubeIDE
3. Build the project, and click Run or Debug to flash the firmware to GH1-a.
4. The firmware can be uploaded using either an official ST-Link programmer or a compatible ST-Link V2 clone.



Pinout and Configuration



Pin Number	Pin Name	Pin Type	Alternate Function(s)	Label
6	VBAT	Power	-	-
16	VSS	Power	-	-
17	VDD	Power	-	-
23	PH0-OSC_IN	I/O	RCC_OSC_IN	-
24	PH1-OSC_OUT	I/O	RCC_OSC_OUT	-
25	NRST	Reset	-	-
30	VDD	Power	-	-
31	VSSA	Power	-	-
32	VREF+	Power	-	-
33	VDDA	Power	-	-
38	VSS	Power	-	-
39	VDD	Power	-	-
40	PA4	I/O	DAC_OUT1	-
41	PA5	I/O	SPI1_SCK	-
43	PA7	I/O	SPI1_MOSI	-
44	PC4	I/O	GPIO_Output	SPI_CS
45	PC5	I/O	GPIO_Output	SPI_DC
46	PB0	I/O	GPIO_Output	SPI_RESET
51	VSS	Power	-	-
52	VDD	Power	-	-
61	VSS	Power	-	-
62	VDD	Power	-	-
71	VCAP_1	Power	-	-
72	VDD	Power	-	-
83	VSS	Power	-	-
84	VDD	Power	-	-
94	VSS	Power	-	-
95	VDD	Power	-	-
98	PC8	I/O	SDIO_D0	-
103	PA11	I/O	USB_OTG_FS_DM	-
104	PA12	I/O	USB_OTG_FS_DP	-

Pin Number	Pin Name	Pin Type	Alternate Function(s)	Label
106	VCAP_2	Power	-	-
107	VSS	Power	-	-
108	VDD	Power	-	-
110	PA15	I/O	GPIO_Input	SD_DETECT
113	PC12	I/O	SDIO_CK	-
116	PD2	I/O	SDIO_CMD	-
118	PD4	I/O	GPIO_Input	btn0
120	VSS	Power	-	-
121	VDD	Power	-	-
122	PD6	I/O	GPIO_Input	btn1
124	PG9	I/O	GPIO_Input	btn2
126	PG11	I/O	GPIO_Input	btn3
130	VSS	Power	-	-
131	VDD	Power	-	-
133	PB3	I/O	GPIO_Input	btn4
138	BOOT0	Boot	-	-
143	PDR_ON	Reset	-	-
144	VDD	Power	-	-

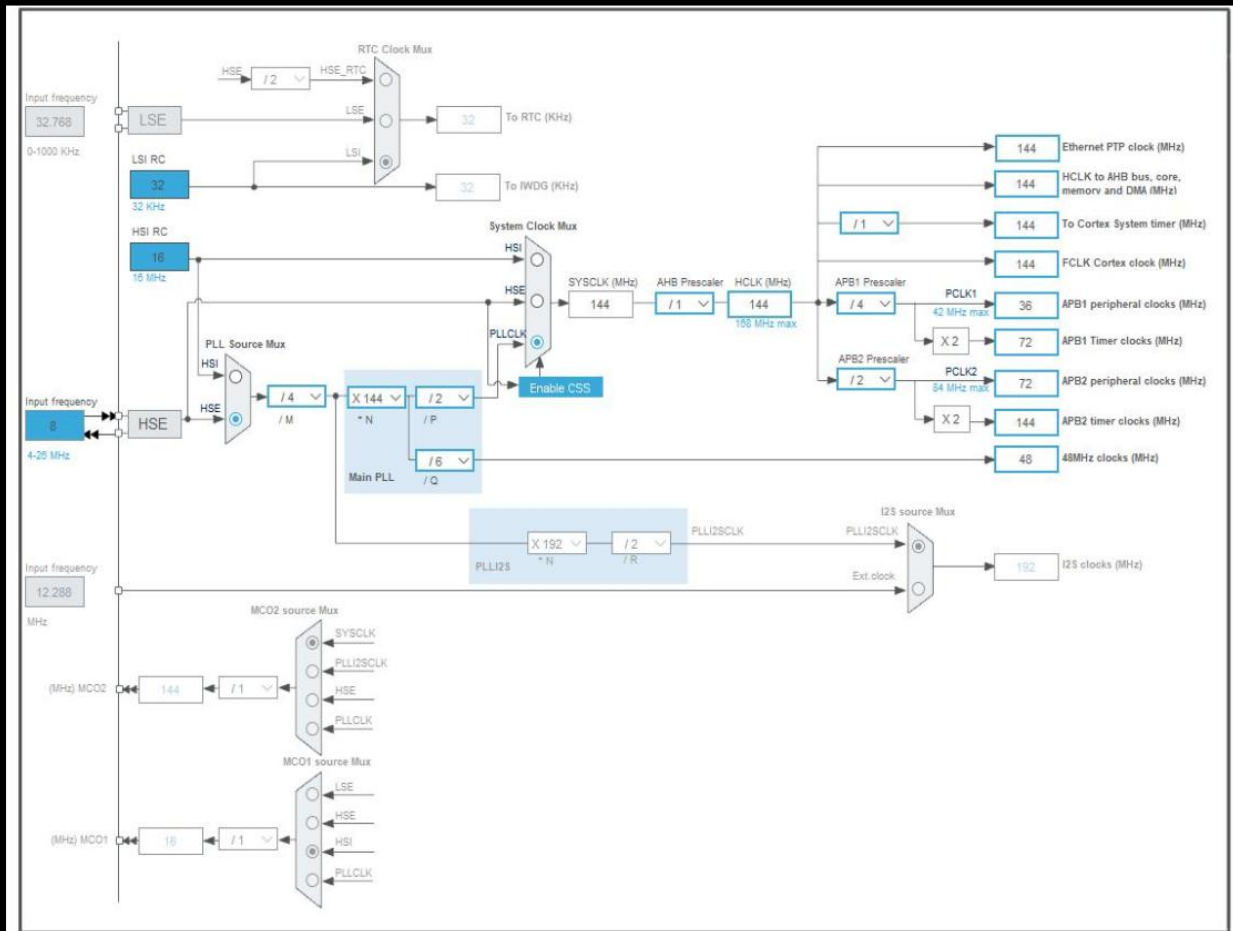
System Configuration

IP	Pin	Signal	GPIO Mode	Pull-Up / Pull-Down	Max Speed	User Label
DAC	PA4	DAC_OUT1	Analog Mode	No Pull-Up / No Pull-Down	N/A	-
RCC	PH0-OSC_IN	RCC_OSC_IN	N/A	N/A	N/A	-
RCC	PH1-OSC_OUT	RCC_OSC_OUT	N/A	N/A	N/A	-
SDIO	PC8	SDIO_D0	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
SDIO	PC12	SDIO_CK	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
SDIO	PD2	SDIO_CMD	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
SPI1	PA5	SPI1_SCK	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
SPI1	PA7	SPI1_MOSI	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
USB_OTG_FS	PA11	USB_OTG_FS_DM	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
USB_OTG_FS	PA12	USB_OTG_FS_DP	Alternate Function Push Pull	No Pull-Up / No Pull-Down	Very High	-
GPIO	PC4	GPIO_Output	Output Push Pull	No Pull-Up / No Pull-Down	High	SPI_CS

IP	Pin	Signal	GPIO Mode	Pull-Up / Pull-Down	Max Speed	User Label
GPIO	PC5	GPIO_Output	Output Push Pull	No Pull-Up / No Pull-Down	High	SPI_DC
GPIO	PB0	GPIO_Output	Output Push Pull	No Pull-Up / No Pull-Down	Low	SPI_RESET
GPIO	PA15	GPIO_Input	Input Mode	Pull-Down	N/A	SD_DETECT
GPIO	PD4	GPIO_Input	Input Mode	Pull-Down	N/A	btn0
GPIO	PD6	GPIO_Input	Input Mode	Pull-Down	N/A	btn1
GPIO	PG9	GPIO_Input	Input Mode	Pull-Down	N/A	btn2
GPIO	PG11	GPIO_Input	Input Mode	Pull-Down	N/A	btn3
GPIO	PB3	GPIO_Input	Input Mode	Pull-Down	N/A	btn4

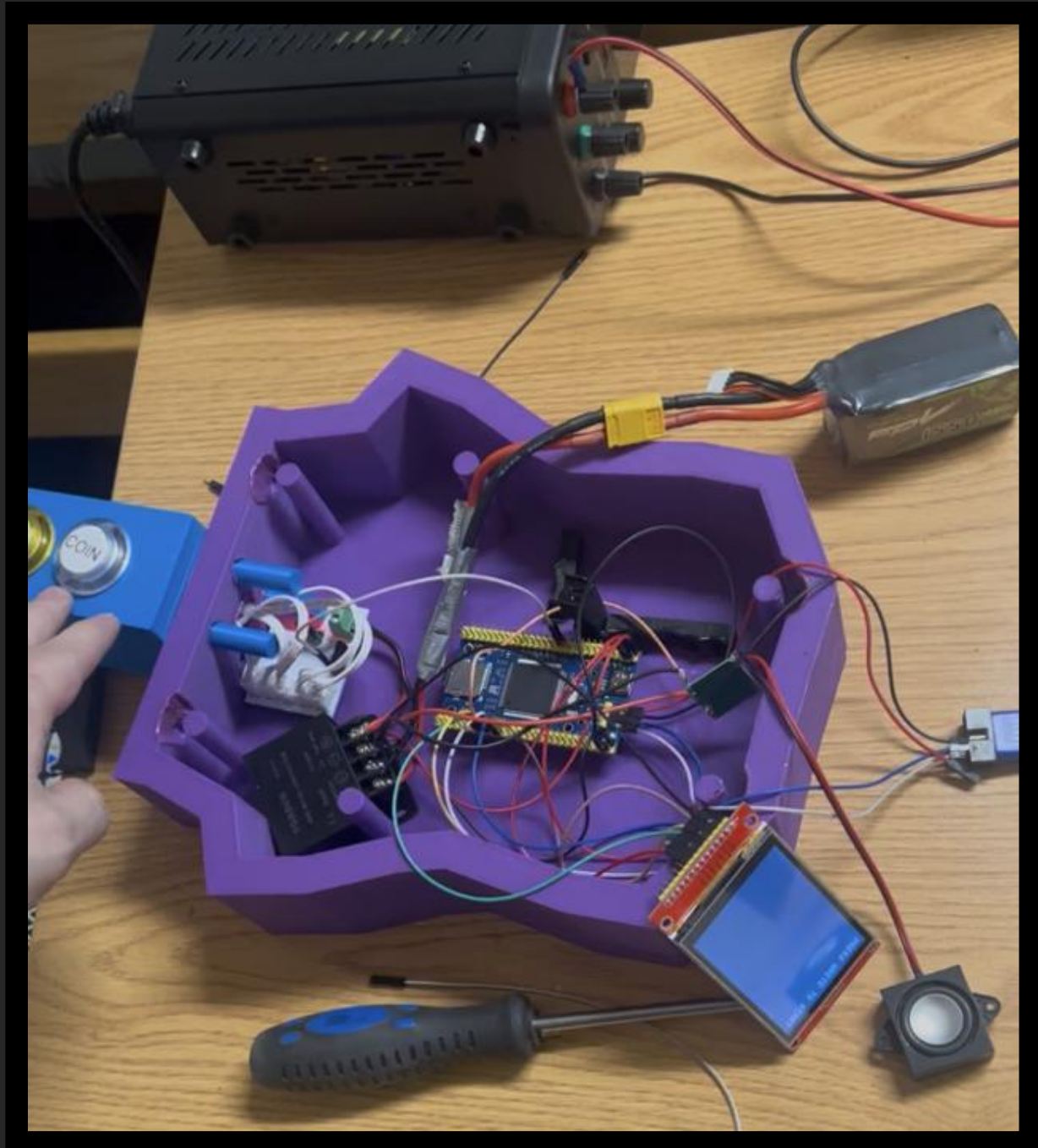
Clock Configuration

GH1-a uses an 8 MHz external crystal (HSE) with the PLL configured to produce a 144 MHz system clock. The AHB bus runs at 144 MHz, APB1 at 36 MHz, and APB2 at 72 MHz, with Flash latency set to four wait states for stable operation. SPI1 is driven by the 72 MHz APB2 peripheral clock and, with a baud rate prescaler of 2, so it halves it and communicates with the display at 36 MHz.

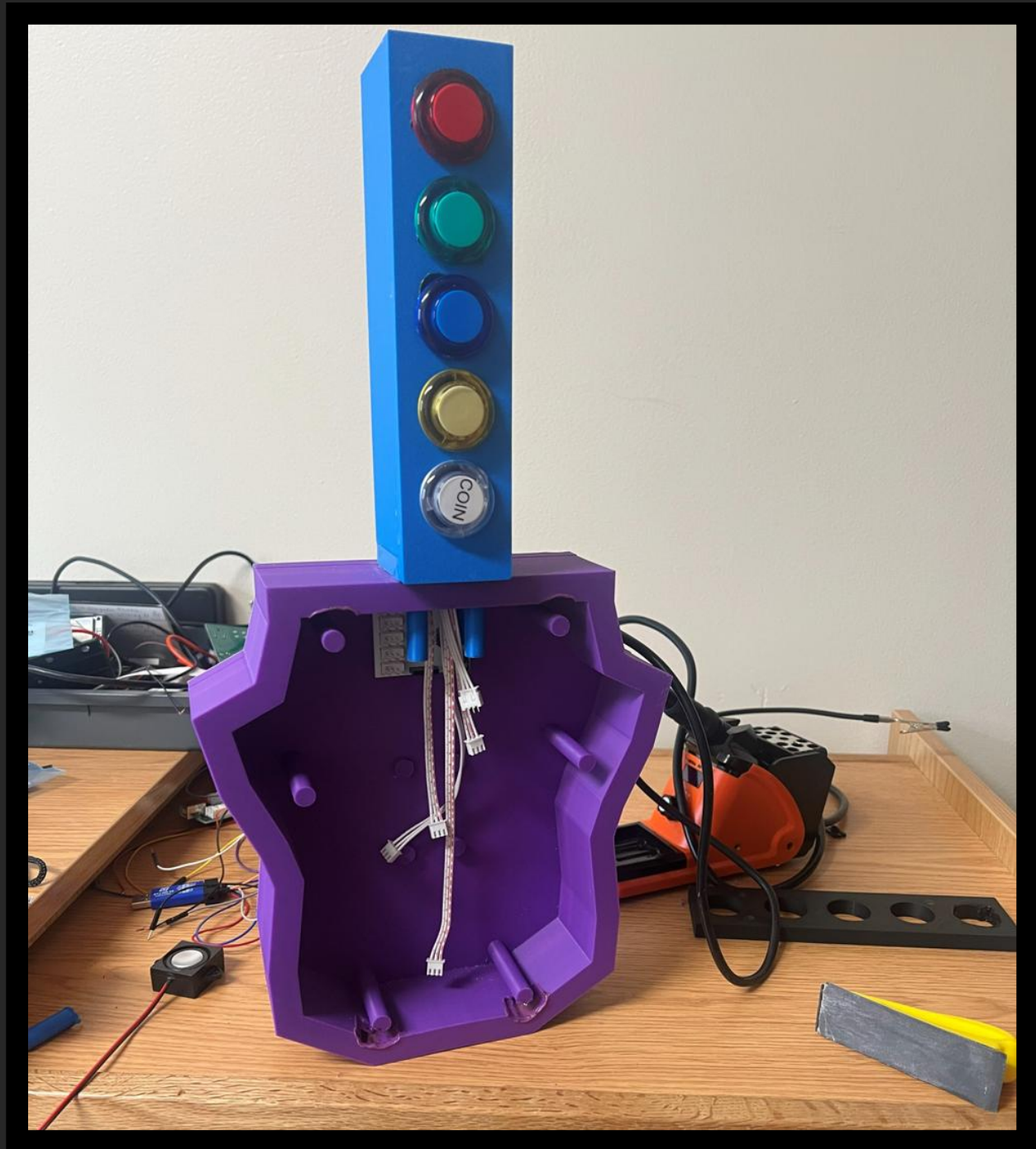


Physical Internal Layout

Note the intentional melting is due to early design flaws and limited access to reprints. The CAD files have been corrected several times after these early prints. The buck converter is placed in the top left corner and I used two scrap pieces glued to the body as a bracket for holding the battery in place. You can see the bottom left rod of the Neck Body Union Piece also has to be removed because it intersects GH1-b. The first 2 photos were taken before the XT60 Switch was added. You can also see how 10.7mm holes are drilled for the Union Piece







Song and MicroSD Card

The microSD card needs to be formatted to these specifications:

Capacity: 64GB (Doesn't matter)

Partition Scheme: MBR (Master Boot Record)

Filesystem: FAT32

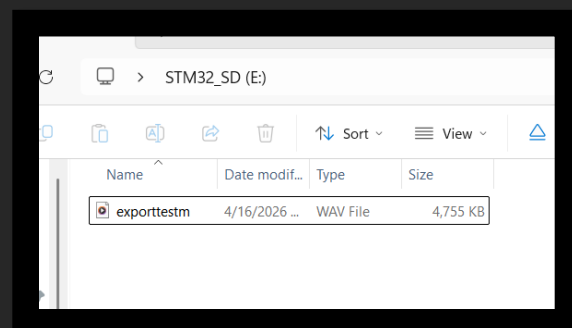
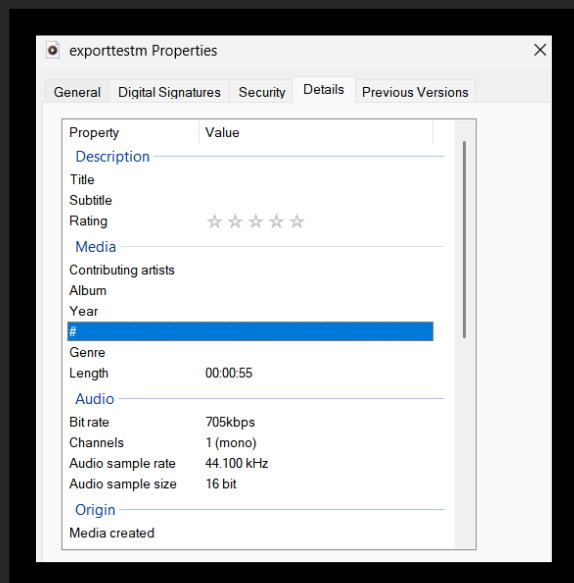
Cluster Size: 32 KB

The .wav file “exporttestm.wav” (I forgot to rename this in the code) should be the only file on the card

The .wav file “exporttestm.wav” written to the microSD is mono-channel, has a 44.1khz sample rate, and an audio sample rate of 16bits.

GH1-a interacts with the microSD from the onboard slot with the SDIO peripheral and the FatFS library enabled for abstracting the process of reading data from the card.

I made the song in LMMS, a digital audio workstation application program. It is an arcade-style adaptation of *End Of Line* by Daft Punk from the movie *Tron Legacy*



Software Architecture

Code is written in C on STM32CubeIDE. STM32 HAL Libraries abstract low level programming, provides framework and handles hardware initialization. GH1s' source code specifies the peripherals GPIO, SPI on channel 1, SDIO with FatFS enabled, DAC1, and a circular DMA Channel that manages an audio buffer. Audio data is also read from the MicroSD using the SDIO peripheral and DMA. DMA Channel 1 is ran at 44.1khz controlled by the timer 7 trigger. The DAC outputs the PCM waveform to pin A4.

A Main function initializes the game state, then goes into a forever loop where it polls for inputs (No interrupts are used), manages and updates the internal game state, and then the game makes draw calls using the ILI9341 driver, in that exact order.

At the game initialization it also starts the audio playback, and over time the visual tempo of the game is altered to correct de-sync issues with the song. I plan to correct the speed of the game and get rid of this due to the code that changes the speed during gameplay being overly complex.

